

City Food Corner: A Desktop-Based Cafe Sales, Inventory, and Workforce Management System

Opu Sutradhar

Department of Computer Science and Engineering
City University, Dhaka, Bangladesh
e-mail: opusutradhar77@gmail.com

Abu Huraira Anik

Department of Computer Science and Engineering
City University, Dhaka, Bangladesh
e-mail: anik090110.islam7.6.5@gmail.com

Md. Roisuddin Rasel

Department of Computer Science and Engineering
City University, Dhaka, Bangladesh
e-mail: mdrasalislam10@gmail.com

Mst. Maimoona

Department of Computer Science and Engineering
City University, Dhaka, Bangladesh
e-mail: maimoonahossain994@gmail.com

Mst. Shikha Moni

Dept. of Computer Science and Engineering
City University, Dhaka, Bangladesh
e-mail: shikha.moni.43.23@gmail.com

Abstract

This paper presents City Food Corner, a desktop-based cafe management system built with Java, JavaFX, and MySQL to streamline daily restaurant operations. The application revolves around three clear-cut user roles: Administrator, Inventory Manager, and Salesman, ensuring staff only see the tools they actually need. Onboarding begins when a prospective employee submits their contact details, password, and target role. The profile remains on hold until the administrator checks the request, confirms it, and issues a permanent employee ID. From there, access splits naturally based on job responsibilities. An Inventory Manager can catalog, update, or search food items, while a Salesman quickly builds customer orders, records payment details, and prints receipts. To keep things clean during busy counter shifts, items that run out of stock drop off the sales interface automatically. Meanwhile, the administrator retains complete control through shift-based attendance tracking, expense ledgers, monthly cash-flow charts, read-only inventory oversight, and a transaction history backed by practical sales analytics. Under the hood, the system is organized cleanly using the Model-View-Controller pattern, leaning on a dedicated Data Access Object layer to run safe, isolated JDBC queries against a local MySQL database. Across twelve functional screens, we walked through twelve real-world operational scenarios by hand (covering everything from registration and role-based login to checkout routines and payroll math), and each test case passed without issue. Right now, the main limitation is that the software is tied to a single desktop workstation. Adding local network synchronization, cloud backups, and digital payment integrations remains our top priority for upcoming releases.

Keywords—Cafe Management System, JavaFX, MySQL, Role-Based Access Control, Inventory Tracking, Point of Sale, Model-View-Controller, Shift Tracking

I. INTRODUCTION

Running a small cafe might seem straightforward to an outside observer, but anyone who has managed one will tell you that the daily reality is quite different. Stock levels need to be monitored so the kitchen does not run out of popular items during peak hours. Every sale has to be recorded correctly to keep the accounts straight at the end of the day. Employees have to check in and check out so the owner knows who worked and for how long. Customer information should be captured so that the business can build a reliable purchase history. Handling all of these tasks through paper notebooks, loose spreadsheets, or memory alone invites mistakes, and when those mistakes pile up, the cafe's overall efficiency and revenue suffer.

In Bangladesh, most small and medium-sized food businesses still keep their records manually or through separate, disconnected tools. Inventory figures sit in one register, sales numbers in another, and staff attendance in a third. There is no single view that tells the owner exactly how the business performed today, this week, or this month. Whenever the owner needs a summary, someone has to cross-check multiple sources, which is time-consuming and prone to error.

Several software solutions exist that address one or two of these issues. Some focus on billing and receipt generation; others handle employee attendance through hardware like

NFC scanners. But many of them either require a continuous internet connection, depend on third-party cloud platforms, or cover only a narrow slice of the daily workflow. For a small cafe that wants a practical, offline, all-in-one solution running on a single computer, the available options are limited.

That observation is basically why City Food Corner exists. We set out to build a system that a small cafe could install on one PC and use right away, without needing the internet or any special devices. The application is written in Java with JavaFX handling the interface and MySQL storing the data, and it follows an MVC layout with a separate data-access layer underneath. This paper walks through the objectives, the related work we studied, the architecture we chose, the screens we built, and the test results we got.

II. OBJECTIVE

We had five main goals going into this project:

1. To build a working cafe management system capable of handling employee access control, inventory operations, sales processing, and cash record-keeping in one place.
2. To replace disconnected manual registers and spreadsheets with a single, unified desktop application.
3. To enforce role-based access so that administrators, inventory managers, and salesmen each work only within their assigned scope.

4. To gain practical experience in Java, JavaFX, database design, and MVC-based application development through a real-world project.
5. To reduce errors in stock counting and recording errors that happen when everything is done by hand.

III. LITERATURE REVIEW

We went through a number of published papers before writing any code, partly to avoid reinventing things and partly to figure out where our project could add something new. What follows is a summary of the work that shaped our thinking the most.

On the restaurant-management side, Pasalkar et al. [1] outlined a framework covering order processing, inventory, and staff scheduling, while Marulkar et al. [2] went a step further and actually built a cloud-integrated POS using PHP and MySQL, complete with digital receipts. Both papers were useful for understanding the typical feature set, but neither one included the kind of employee-approval workflow or shift-tracking module that we wanted. The POS system in [2] also depends on cloud hosting, which was something we specifically wanted to avoid.

Shimmura et al. [3] took a different angle altogether. They focused on what happens when food-order data is shared in real time across different stations in a restaurant and showed measurable improvements in service speed. That idea stuck with us. We made sure that in City Food Corner, stock counts and cash totals update the moment a sale goes through, so nobody is ever looking at stale numbers. Abel and Ekabua [4], on the other hand, explored customer self-ordering to cut wait times. Their approach suits large self-service venues, but in a typical Bangladeshi cafe, the salesman handles everything at the counter, so we kept the salesman role central to our checkout flow.

For attendance tracking, Oo et al. [5] described a system that uses NFC cards and a cloud backend. It works, but it means buying NFC readers and paying for server space. We went with a simpler method: when an employee logs in, the system records their check-in time automatically, and when they log out, it calculates the hours. No extra hardware needed.

Sarker and Apu [6] wrote about applying MVC to Java desktop apps, which was probably the most directly relevant paper for our architecture decisions. We followed a similar separation of concerns and added a DAO layer on top of it to keep database queries out of the controller classes. Then et al. [7] built a web-based payroll system, and while it is web-based rather than desktop, their financial-reporting module gave us ideas for our own cash-flow screen, where salary payments, inventory expenses, and personal withdrawals all feed into one dashboard.

The access-control design leans heavily on the RBAC model introduced by Ferraiolo and Kuhn [8] and later formalized by Sandhu et al. [9]. In practice, this means that once you log in, you only see the screens that match your role nothing more, nothing less. We also looked at Erdem and Sandikkaya's OTPaaS framework [10] for two-factor authentication but did not implement it in this version; it remains on the list for future work.

Looking at all of these together, the pattern was pretty

clear: most existing systems solve one or two parts of the problem well but leave the rest untouched. We wanted to bring employee management, stock control, sales, attendance, and cash-flow reporting under one roof, and that is where City Food Corner fits in.

IV. PROBLEM STATEMENT

Walk into most small cafes in Dhaka and you will find the same setup: a notebook for orders, a separate register for stock, maybe a third one for attendance, and a calculator for the day's totals. Each of these records lives in its own little world. If someone miscounts a stock item or forgets to mark a sale, the error only surfaces when the owner sits down at night to reconcile the numbers by which point it is almost impossible to trace back to the source.

The digital tools that do exist tend to tackle only one piece of the puzzle. A billing app handles receipts but knows nothing about inventory. An attendance system tracks clock-ins but cannot tell you how much a particular employee sold that day. And most of these tools need either a constant internet connection or specialized hardware, which pushes the cost beyond what a small cafe can justify. What we needed, but could not find, was a single offline desktop application that covered employee management, inventory, sales, and financial reporting without extra hardware or cloud subscriptions.

V. PROPOSED METHODOLOGY

We developed the system using an iterative, modular approach, building the four core modules (authentication, administration, inventory management, and sales processing) one at a time. After testing each module on its own, we integrated it with the rest and re-tested the earlier components to make sure nothing broke. The database schema was not set in stone from day one; it evolved naturally alongside development. Whenever a new feature called for data we had not originally planned for, we updated the schema with additional fields or tables and re-verified that everything still functioned smoothly.

The core operating flow, illustrated in Fig. 1, begins when a new staff member registers with their name, email, phone number, password, and requested role. The system checks for duplicate emails, and if the address is unique, it creates the account in a pending state. From there, the administrator reviews the request and either approves it by assigning a permanent employee ID (e.g., CFCE-001), or blocks or deletes it entirely.

Once approved, employees can log in using their credentials and assigned role. At the moment of login, the system automatically records a check-in timestamp for that day's shift. Depending on the role selected, the employee is directed either to the inventory workspace to add, update, search, or remove products, or to the sales workspace to pick items from the live catalog, enter customer details, and generate receipts upon checkout. Every action seamlessly updates the administrator's dashboard, cash-flow ledger, transaction history, and attendance records. When the employee hits the exit button at the end of their shift, the system logs their checkout time and automatically computes their total working hours.

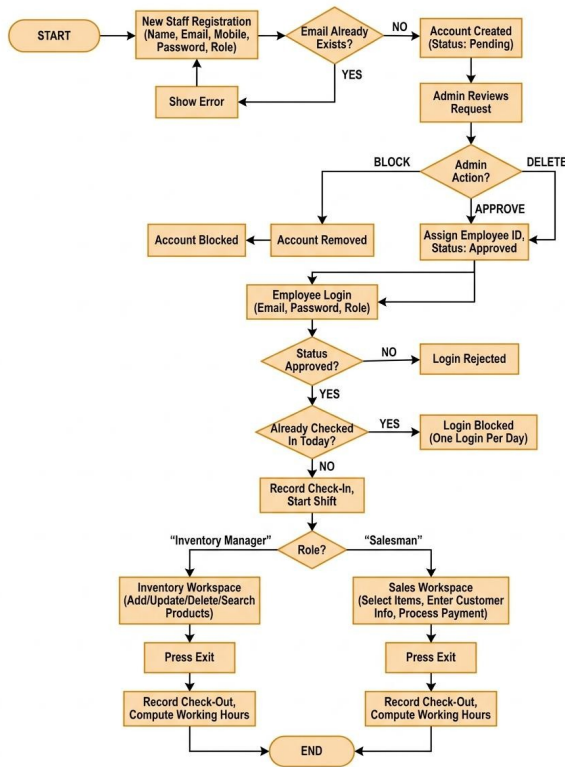


Fig. 1. Flowchart of City Food Corner's operating logic, from account registration through role-based login to inventory or sales completion.

Each module was built and tested individually before being integrated into the complete application, and the database schema was refined alongside the modules as new fields or relationships were found to be necessary during development.

VI. SYSTEM ARCHITECTURAL FRAMEWORK DESIGN

A. Architecture

City Food Corner is structured around the Model-View-Controller (MVC) architecture, with a dedicated DAO layer bridging the models and the database, as illustrated in Fig. 2. At the top, the View layer consists of eleven JavaFX screens defined through FXML markup paired with CSS for styling. Each view has a matching Controller that handles button clicks, form submissions, and user interactions. These controllers rely on a set of shared utility classes, including SceneSwitcher for fluid screen transitions, SessionManager for keeping track of active sessions and user roles, PasswordUtil for secure SHA-256 password hashing, and lightweight helpers for search filtering and input validation.

Directly below the controllers lies the Model layer, built from plain Java classes representing core business entities such as Employee, Product, OrderItem, HistoryRecord, and ShiftRecord. These classes carry data back and forth without containing any user-interface logic. By implementing JavaFX observable properties directly inside these models, our TableView components and controls can bind directly to the data, which eliminated the need for tedious manual refresh routines across the application.

Persistence is handled by a separate Data Access Object (DAO) layer composed of five specialized classes: EmployeeDAO, ProductDAO, SaleDAO, ShiftDAO, and

CashFlowDAO. Each DAO encapsulates the raw JDBC operations and SQL queries required to interact with the city_food_corner MySQL database. Isolating the database calls entirely within this layer proved to be a practical design choice during development, as queries could be tweaked or rewritten without breaking or needing changes in the controller and screen logic.

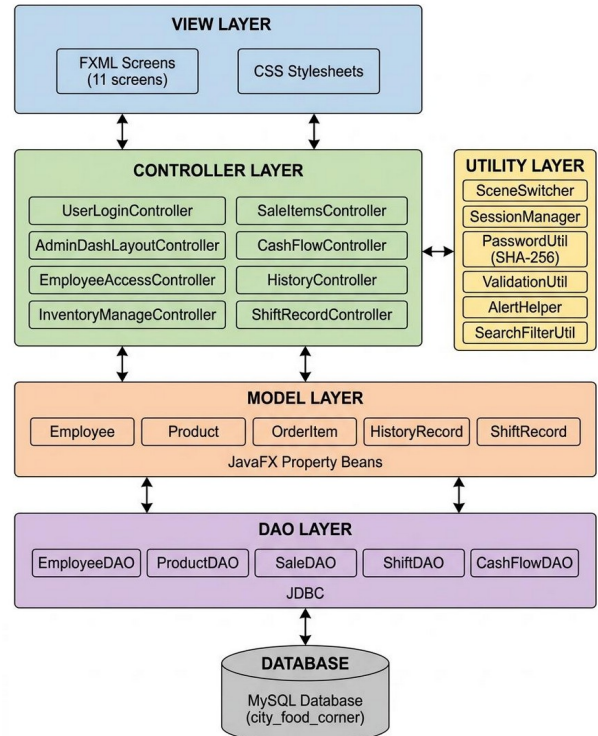


Fig. 2. High-level MVC architecture of City Food Corner, showing the View (FXML + CSS), Controller, Utility, Model, and DAO layers connecting to the MySQL database.

B. Database Design

The MySQL database, named city_food_corner, runs on eight tables linked together as shown in the ER diagram in Fig. 3. The EMPLOYEES table stores basic staff details, hashed passwords, roles, and approval statuses. We added a simple boolean flag called id_locked so that once an admin assigns an employee code, nobody can accidentally change it later. Directly linked to this is SHIFT_RECORDS, which keeps a clean log of every check-in, check-out, and shift date for attendance.

For daily operations, PRODUCTS holds the food catalog, including item types, live stock counts, prices, and image paths. When someone makes a purchase, the SALES table records the big picture—customer details, the bill, change returned, who handled the counter, and the exact timestamp. Meanwhile, SALE_ITEMS stores the individual items on that receipt, tied to both the sale (with cascading delete enabled) and the product list.

The last three tables keep track of the money. CASH_EXPENSES tracks day-to-day running costs by category and date, while CASH_RESERVE watches the net funds on hand. Lastly, SALARY_CONFIG is a tiny, single-row table we added to remember the last-entered salary figures, saving the admin from having to type them in from scratch every time they open the cash-flow screen.

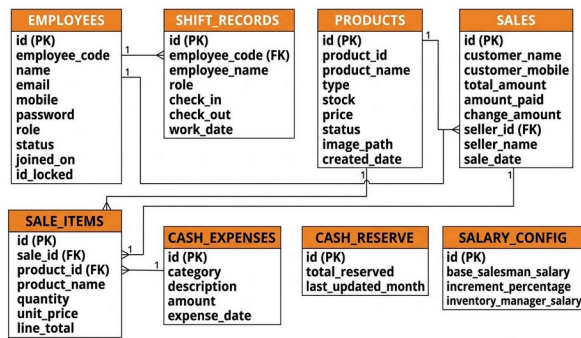


Fig. 3. Entity-Relationship diagram of the city_food_corner database (eight tables).

C. Tools and Technologies

1. Programming Language: Java (JDK 17)
2. GUI Framework: JavaFX 21 with FXML and CSS
3. Database: MySQL 8.0
4. Architecture Pattern: Model-View-Controller (MVC) with a dedicated DAO layer
5. Build Tool: Apache Maven
6. Development Environment: IntelliJ IDEA

VII. IMPLEMENTATION AND RESULTS

We ended up with twelve working screens. Below is a quick walkthrough of each one, roughly in the order a user would encounter them.

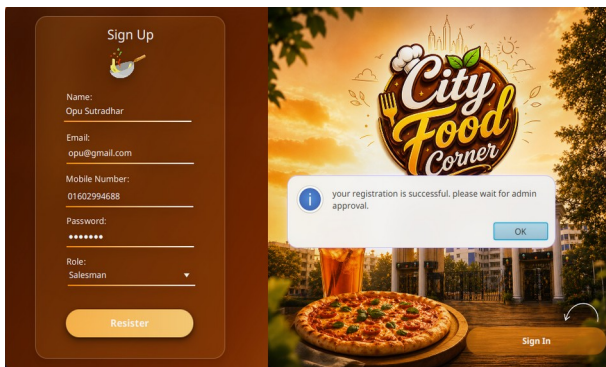


Fig. 4. Sign-up Screen: New staff sign up with their details and select a role (Salesman or Inventory Manager). The system checks for missing fields and duplicate emails, then holds the account in a pending state until an admin approves it.



Fig. 5. Employee login : An approved staff member signs in with their email, password, and role to reach a role-specific workspace, with a link back to the sign-up page for new staff.



Fig. 6. Forgot-password dialog: The employee confirms their identity using the email and phone number on file.



Fig. 7. Administrator login: Administrators log in through a separate, fixed-credential portal isolated from regular staff sign-in, keeping the account-control panel secure.

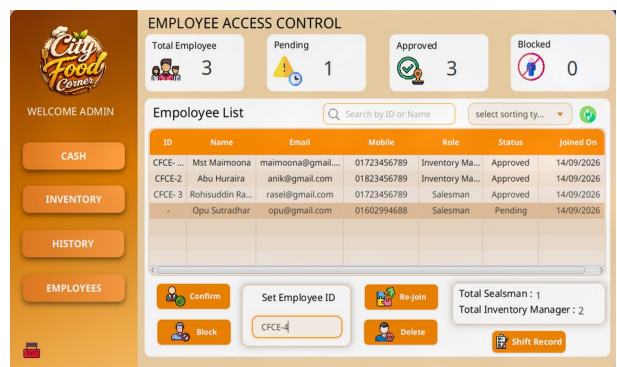


Fig. 8. Employee Access Control: The admin can view total, pending, approved, and blocked employee counts alongside the full staff list. From here, they can confirm pending accounts with a fixed ID, block users, re-join them, or delete profiles entirely.



Fig. 9. Shift Record: logs check-in, check-out, and total working hours for every employee. Records can be filtered by date, role, and report type to simplify attendance tracking and payroll.

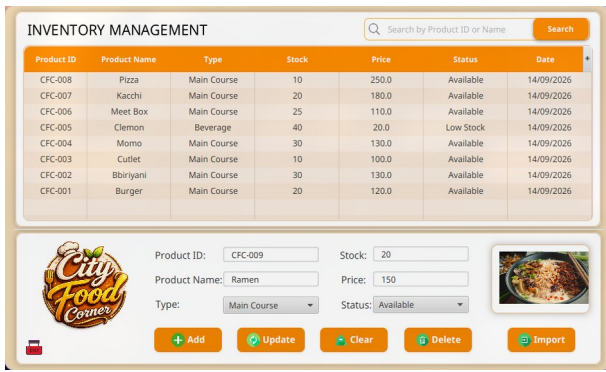


Fig. 10. Inventory Management screen (Inventory Manager): Displays all products with their stock, price, type, and status. It allows the inventory manager to add, update, delete, search, or import items to keep stock records accurate in real time.

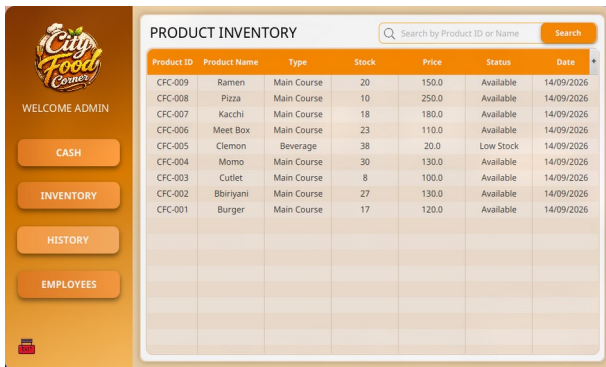


Fig. 11. Administrative inventory view: Gives the owner a read-only, searchable record of every product the inventory manager has added or updated. This lets the owner monitor stock changes without risk of accidental edits.

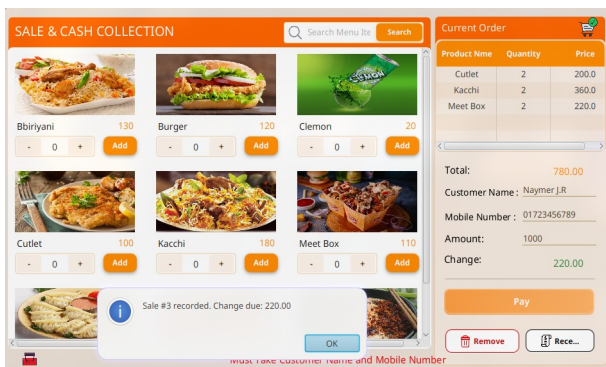


Fig. 12. Sale and Cash Collection screen (Salesman): The salesman builds a running order from the live menu while the system automatically computes the total, accepts the amount paid, and calculates change. This streamlines checkout and reduces manual calculation errors during sales.

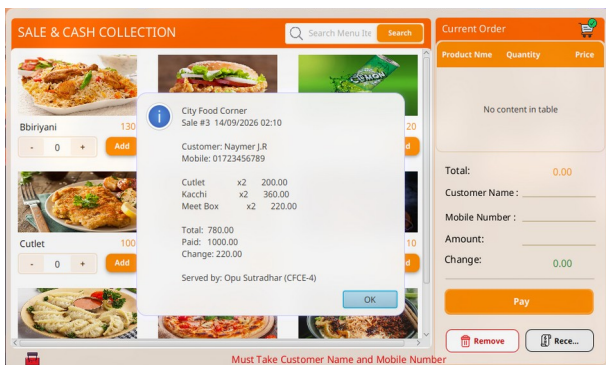


Fig. 13. Digital receipt: Pops up right after checkout, laying out the purchased items, customer name and phone number, total bill, amount paid, change due, and the salesman who handled the counter

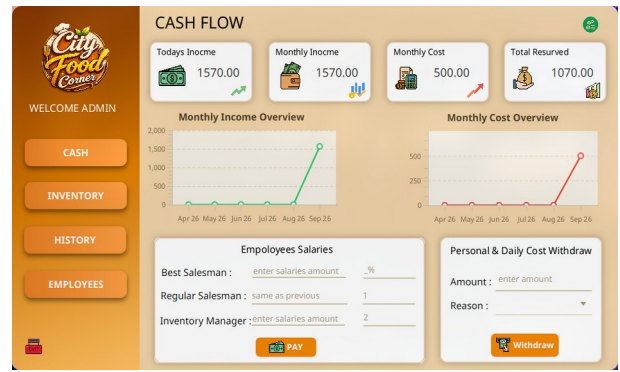


Fig. 14. Cash Flow screen (Administrator): Shows today's and monthly income, monthly cost, and total reserved funds, with trend charts, salary payment controls, and a personal/daily cost withdrawal panel.



Fig. 15. History screen (Administrator): Logs every completed sale with customer details, items purchased, amount, date, and seller ID, alongside a customer-traffic chart, top-selling items, and the best salesman.

VIII. TESTING

Since this is a small-scale academic project running on a single machine, we tested everything by hand rather than writing automated scripts. We designed twelve scenarios that cover the most important workflows and ran each one manually. Table I shows what we tested and what happened.

Table I. Functional test scenarios and results.

Scenario	Result	Status
Sign-up, login before approval	Rejected; stays 'Pending'	Pass
Admin confirms pending account	Approved; login allowed	Pass
Duplicate check-in, same day	Second check-in blocked	Pass
Admin blocks an employee	Login rejected till re-joined	Pass
Stock reduced to zero	Item hidden from sale menu	Pass
Multi-item order + payment	Total / change computed right	Pass
Receipt after a sale	Items, total, change, seller ID correct	Pass
Sale vs. History / Dashboard	New entry + totals update live	Pass
Exit after check-in	Shift closed; hours computed	Pass
Pay salary from Cash Flow	Deducted from monthly cost	Pass
Withdraw personal/daily cost	Deducted from reserved funds	Pass
Search by ID/name	Rows filtered correctly	Pass

IX. LIMITATIONS

A few practical limitations in the current version of City Food Corner include:

1. The system checks email format locally but lacks live verification, leaving it unable to confirm if an email

address actually exists.

2. The system currently supports only cash-based transactions and does not include online payment options such as mobile financial services or card payments.
3. The database runs locally, which restricts the application to a single device and prevents real-time multi-device access.
4. There is no automated backup facility at present, so data may be lost if the host machine fails or the database becomes corrupted.

X. FUTURE WORK

To build on the current system, we hope to focus on the following future improvements:

1. Migrating the local MySQL database to a cloud-hosted instance so that the system can be accessed from more than one device at the same time.
2. Adding two-factor authentication with a one-time password (OTP) mechanism, similar to the OTP-as-a-service approach in [10], to strengthen login security.
3. Adding support for mobile financial services such as bKash and Nagad, along with card payments, so that customers are not limited to cash transactions.
4. Adding email verification at the time of registration, so that only genuine email addresses are accepted into the system.
5. Adding a scheduled backup mechanism for the local database to reduce the risk of data loss.

XI. CONCLUSION

We set out to solve a very real headache for small food businesses: having to juggle daily tasks across scattered spreadsheets, paper logs, or costly subscription tools. City Food Corner brings everything (staff onboarding, inventory control, cashier checkout, shift hours, and cash-flow tracking) under one roof. Powered by Java, JavaFX, and a local MySQL database, the app sticks to a clean MVC pattern with a dedicated DAO layer, running entirely offline on a single machine without needing extra hardware or constant internet access.

Across twelve working screens, we walked through twelve hands-on operational scenarios, covering everything from signing up new staff to closing out shifts and printing receipts. Every single test run passed without issue. The software certainly has its limits in its current form: it is tied to one device, does not handle online transactions, and lacks cloud backups.

Even so, the implementation validates that a lightweight, role-based desktop system can successfully integrate and streamline core point-of-sale, inventory, and staff management tasks for small food businesses without requiring complex infrastructure.

XII. REFERENCES

- [1] O. Pasalkar, N. Ahire, V. Hande, R. Paikar, H. Hatkar, and S. Deshpande, "Restaurant Management System," *Int. Res. J. Modernization Eng. Technol. Sci.*, vol. 6, no. 1, pp. 3323–3326, Jan. 2024.
- [2] P. Marulkar, V. Tandel, M. Thakur, and G. Thakur, "Design and Implementation of a Cloud-Integrated Restaurant Point of Sale (POS) System," *Int. J. Res. Publ. Rev.*, vol. 6, no. 4, pp. 5754–5758, Apr. 2025.
- [3] T. Shimmura, T. Takenaka, and M. Akamatsu, "Real-time process management system in a restaurant by sharing food order information," in *Proc. 2009 Int. Conf. Soft Comput. Pattern Recognit. (SoCPaR)*, Malacca, Malaysia, 2009, pp. 703–706, doi: 10.1109/SoCPaR.2009.141.
- [4] E. E. Abel and O. O. Ekabua, "Restaurant Customer Self-Ordering System: A Solution to Reduce Customer/Guest Waiting Time at the Point of Sale," *Int. J. Comput. Appl.*, vol. 111, no. 11, pp. 19–22, Feb. 2015, doi: 10.5120/19589-1393.
- [5] S. B. Oo, N. H. M. Oo, S. Chainan, A. Thongniam, and W. Chongdarakul, "Cloud-based web application with NFC for employee attendance management system," in *Proc. 3rd Int. Conf. Digital Arts, Media and Technology (ICDAMT)*, Phayao, Thailand, 2018, pp. 163–167, doi: 10.1109/ICDAMT.2018.8376516.
- [6] I. H. Sarker and K. Apu, "MVC Architecture Driven Design and Implementation of Java Framework for Developing Desktop Application," *Int. J. Hybrid Inf. Technol.*, vol. 7, no. 5, pp. 317–322, Sep. 2014, doi: 10.14257/ijhit.2014.7.5.29.
- [7] H. J. Then, B. Issac, L. T. Chung, S. Wongso, R. Bong, L. Lu, and Y. P. Ling, "Analysis, Design and Implementation of a Web-based Payroll Application Software," in *Proc. 2009 Int. Conf. Comput. Technol. Dev. (ICCTD)*, Kota Kinabalu, Malaysia, 2009, vol. 2, pp. 107–111, doi: 10.1109/ICCTD.2009.183.
- [8] D. F. Ferraiolo and D. R. Kuhn, "Role-Based Access Controls," in *Proc. 15th Nat. Comput. Secur. Conf.*, Baltimore, MD, USA, 1992, pp. 554–563.
- [9] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996, doi: 10.1109/2.485845.
- [10] E. Erdem and M. T. Sandikkaya, "OTPaaS—One Time Password as a Service," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 3, pp. 743–756, Mar. 2019, doi: 10.1109/TIFS.2018.2866332.